

Examen Calcul Numeric — Set 2 — 14 iunie (restanță)

Student: *Petec Răzvan-Gabriel*

Grupa: 235

Problema 4. Coeficienții din relația de recurență pentru polinoamele (π_n) ortogonale pe $\mathbb{R}$ în raport cu ponderea $w(t) = \frac{e^{-t}}{(1+e^{-t})^2}$ sunt $\alpha_k = 0$, $\beta_0 = 1$, $\beta_k = \frac{k^4 \pi^2}{4k^2 - 1}$.

- (a) Verificați ortogonalitatea polinoamelor (π_n) . (1p)
- (b) Implementați în MATLAB o cuadratură de tip Gauss pentru ponderea și intervalul de mai sus. (1p)
- (c) Calculați

$$\int_{-\infty}^{\infty} w(t) \cos(t) dt$$

cu funcția de la punctul (b) cu o precizie de 10^{-6} . (2p)

Soluție

```
syms t;  
w=exp(-t)/(1+exp(-t))^2
```

$$w = \frac{e^{-t}}{(e^{-t} + 1)^2}$$

Subpunctul (a)

Polinoamele π_k sunt date de relația de recurență: $\pi_{k+1}(x) = (x - \alpha_k)\pi_k(x) - \beta_k\pi_k(x)$, $\pi_{-1}(x) = 0$, $\pi_0(x) = 1$. Din enunț,

$$\pi_{k+1}(x) = x\pi_k(x) + \frac{k^4 \pi^2}{4k^2 - 1} \pi_{k-1}(x), \text{ pentru } k \geq 1, \pi_1(x) = x\pi_0(x) - \pi_{-1}(x) = x$$

rezulta ca:

$$\pi_2(x) = x\pi_1(x) - \pi_0(x) \frac{\pi^2}{3} = x^2 - \frac{\pi^2}{3}$$

```
function pi_1=nth_poly_sym(n, x)  
    if n==-1, pi_1=0; return; end  
    if n==0, pi_1=1; return; end  
  
    pi_0=1; pi_1=x;  
    for k=1:n-1  
        pi_kn=x*pi_1 - (k^4*sym(pi)^2)/(4*k^2-1)*pi_0;
```

```

        pi_0=pi_1;
        pi_1=pi_kn;
    end
end

```

Ponderea $w(t)$ este o functie para $\left(w(-t) = \frac{e^t}{(1+e^t)^2} = \frac{e^t}{e^{2t}(1+e^{-t})^2} = \frac{e^{-t}}{(1+e^{-t})^2} = w(t) \right)$

```

n=5;
mat_orto=zeros(n, n);
for i=0:n
    pi_i=nth_poly_sym(i,t);
    for j=0:n
        pi_j=nth_poly_sym(j,t);
        mat_orto(i+1,j+1)=vpaintegral(matlabFunction(w*pi_i*pi_j), -inf,
inf);
    end
end
mat_orto

```

```

mat_orto = 6x6
    1.000000000000001    -5.15651487170367e-19    -9.71445146547012e-17 ...
   -5.15651487170367e-19     3.28986813369645    -1.21929822370514e-17
   -9.71445146547012e-17    -1.21929822370514e-17     34.6343434787564
   -7.79961085071805e-18     2.51534904016637e-15    -2.72234124098645e-16
    1.16573417585641e-15     1.59821407779653e-16     6.3948846218409e-14
    5.83468808967341e-17    -3.01980662698043e-14     1.40000135278961e-14

```

Se observa ca poligoanele sunt ortogonale 2 cate 2.

Demonstratie

Fie $i \neq j$. Atunci:

$$\begin{aligned}
 (\pi_i, \pi_j) &= \int_{-\infty}^{\infty} w(t) \pi_i(t) \pi_j(t) dt = \int_{-\infty}^{\infty} w(t) \left(t \pi_{i-1}(t) + \frac{i^4 \pi^2}{4i^2 - 1} \pi_{i-2}(t) \right) \left(t \pi_{j-1}(t) + \frac{j^4 \pi^2}{4j^2 - 1} \pi_{j-2}(t) \right) dt = \\
 &= \int_{-\infty}^{\infty} w(t) t^2 \pi_{i-1}(t) \pi_{j-1}(t) + w(t) t \frac{i^4 \pi^2}{4i^2 - 1} \pi_{i-2}(t) \pi_{j-1}(t) + w(t) t \frac{j^4 \pi^2}{4j^2 - 1} \pi_{j-2}(t) \pi_{i-1}(t) + w(t) \frac{i^4 \pi^2}{4i^2 - 1} \pi_{i-2}(t) \frac{j^4 \pi^2}{4j^2 - 1} \pi_{j-2}(t) dt
 \end{aligned}$$

Observam ca, pentru ca $(\pi_i, \pi_j) = 0$ este necesar ca:

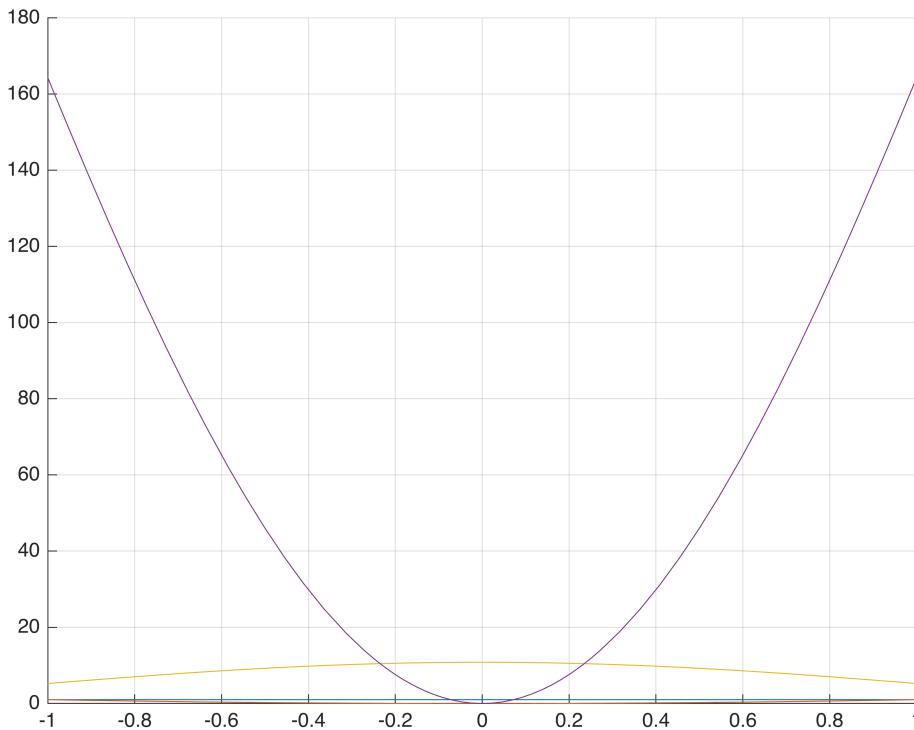
- $\int_{-\infty}^{\infty} w(t) t^2 \pi_{i-1}(t) \pi_{j-1}(t) dt = 0$, ceea ce se poate obtine din recursivitate.
- $\int_{-\infty}^{\infty} w(t) t^2 \pi_{i-2}(t) \pi_{j-2}(t) dt = 0$, ceea ce se poate obtine din recursivitate (inmultind cu o functie para, nu va afecta paritatea).
- $\int_{-\infty}^{\infty} w(t) t \frac{i^4 \pi^2}{4i^2 - 1} \pi_{i-2}(t) \pi_{j-1}(t) + w(t) t \frac{j^4 \pi^2}{4j^2 - 1} \pi_{j-2}(t) \pi_{i-1}(t) dt = 0$. Aici trebuie tratat cazul cand $i = j + 1 / j = i + 1$, altfel se poate obtine din recursivitate.

Pentru cazurile cu recursivitate, problema se reduce la a arata ca $(\pi_0, \pi_1) = (\pi_1, \pi_0) = (\pi_0, \pi_2) = (\pi_2, \pi_0) = 0$, ceea ce se poate observa direct in tabelul aratat anterior.

Pentru cazul $i = j + 1$ (echivalent cu $j = i + 1$), unul dintre membrii se poate reduce, si va ramane integrala:

$$\int_{-\infty}^{\infty} w(t) t \frac{t^4 \pi^2}{4t^2 - 1} \pi_{j-1}(t) \pi_{j-1}(t) dt.$$

```
clf; hold on; grid on;
for j=0:3
    pi_j=nth_poly_sym(j,t);
    fplot(pi_j^2, [-1 1])
end
```



Se observa grafic ca π_k^2 este o functie para (trebuie demonstrat, dar nu e timp), deci integrala mentionata anterior va fi egala cu 0 (deoarece avem par * impar * par = impar).

Deci, in concluzie, polinoamele π_i sunt ortogonale.

Subpunctul (b)

Implementare simbolica:

Avem functia care ne da al k -lea polinom, deci putem sa gasim cuadratura de tip Gauss astfel:

```
function [nodes,coefs]=gauss_poly_quad_sym(n)
syms x;
```

```

w=exp(-x)/(1+exp(-x))^2;
a=-sym('inf'); b=sym('inf');
pin=nth_poly_sym(n, x);
nodes=simplify(solve(pin==0))';
coefs=gauss_coefs_sym(w,a,b,nodes);
end

```

Implementare numerica:

```

function [I,nodes,coefs]=gauss_poly_quad_num(n, f)
alpha=zeros(1,n);
beta=[1 ((1:n-1).^4 * pi^2) ./ (4*(1:n-1).^2 -1)];

J=diag(alpha)+diag(sqrt(beta(2:end)),-1)+diag(sqrt(beta(2:end)),1);
[V,nodes]=eig(J, 'vector');
coefs=beta(1)*V(1,:).^2;
I=coefs*f(nodes);
nodes=nodes';
end

```

Subpunctul (c)

Valoare asteptata

```

exp_quad=vpaintegral(@(x) exp(-x)/(1+exp(-x)).^2 .* cos(x), -inf,
inf, 'RelTol', 1e-12, 'AbsTol', 0)

```

exp_quad = 0.272029054982

Valoare reala

```

f=@(x) cos(x);
I_prev=0; I_crt=0;
for n=1:100
    I_crt=gauss_poly_quad_num(n,f);
    err=abs(I_crt - exp_quad);
    if err<1e-6
        disp("aproximare gasita")
        n, err, I_crt, I_prev
        break
    end
    I_prev=I_crt;
end
end

```

```

aproximare gasita
n =
    20
err = 0.0000005088095067120782907250480371587
I_crt =
    0.272028546172626
I_prev =
    0.272030206151788

```

Observam ca de la 20 de noduri, diferenta absoluta intre valoarea asteptata si cea obtinuta este mai mica decat 10^{-6} , deci pentru minim 20 de noduri vom avea o aproximatie de minim 6 zecimale.

Problema 5. (a) Se consideră ecuația în $\mathbb{R}$ $f(x) = 0$ cu rădăcina α și o metodă cu ordinul de convergență p și eroarea asimptotică C_p . Dacă se fac N_p operații pe pas și operațiile de inițializare se ignoră, arătați că numărul total de operații necesar pentru a aproxima soluția cu precizia ε este

$$T_p = \frac{N_p}{\log p} \log \left[\frac{\frac{\log C_p}{p-1} + \log \varepsilon}{\frac{\log C_p}{p-1} + \log e_0} \right], \quad (**)$$

unde baza logaritmului este arbitrară, e_0 este eroarea inițială. (1p)

(b) Considerăm algoritmul al cărui pas constă din doi pași ai metodei secantei. Care este ordinul de convergență al algoritmului? (1p)

(c) Când este metoda lui Newton mai eficientă decât o metodă cu ordinul de convergență trei? Folosiți formula de la punctul (a). (1p)

Indicație: Fie $e_n = |x_n - \alpha|$ eroarea la pasul n . Se pune $e_{n+1} \approx C_p e_n^p$. Din condiția $e_n \approx \varepsilon$ se scoate n .

Soluție

Subpunctul (a)

Ordinul de convergență: p

Eroarea asimptotică: $C_p = \frac{\phi^{(p)}(\alpha)}{p!}$

Dacă N_p ar fi 1, atunci eroarea s-ar dezvolta astfel per pas:

$$e_{n+1} \sim C_p \cdot e_n^p.$$

Fie n numărul de pași efectuați pentru a se ajunge la precizia dorită ε . Atunci putem găsi formula reducerii erorii ca fiind:

$$e_n \sim C_p^{1+p^2+p^3+\dots+p^{n-1}} \cdot e_0^{p^n} = C_p^{\frac{p^n-1}{p-1}} \cdot e_0^{p^n}.$$

Numărul de pași n este dat de relația $e_n < \varepsilon$. Deci obținem relația:

$$C_p^{\frac{p^n-1}{p-1}} \cdot e_0^{p^n} < \varepsilon$$

Aplicăm un logaritm (cu baza arbitrară) în ambele părți, vom obține:

$$\begin{aligned}
\log\left(C_p^{\frac{p^n-1}{p-1}} \cdot e_0^{p^n}\right) &< \log(\epsilon) \Leftrightarrow \frac{p^n-1}{p-1} \log(C_p) + p^n \log(e_0) < \log(\epsilon) \Leftrightarrow \\
\Leftrightarrow \frac{p^n}{p-1} \log(C_p) + p^n \log(e_0) &< \frac{\log(C_p)}{p-1} + \log(\epsilon) \Leftrightarrow p^n \left(\frac{\log(C_p)}{p-1} + \log(e_0) \right) < \frac{\log(C_p)}{p-1} + \log(\epsilon) \Leftrightarrow \\
\Leftrightarrow p^n &< \frac{\frac{\log(C_p)}{p-1} + \log(\epsilon)}{\frac{\log(C_p)}{p-1} + \log(e_0)}
\end{aligned}$$

Logaritmăm din nou rezultatul obținut anterior. Vom obține:

$$n \log(p) < \log\left[\frac{\frac{\log(C_p)}{p-1} + \log(\epsilon)}{\frac{\log(C_p)}{p-1} + \log(e_0)} \right] \Leftrightarrow n < \frac{1}{\log(p)} \log\left[\frac{\frac{\log(C_p)}{p-1} + \log(\epsilon)}{\frac{\log(C_p)}{p-1} + \log(e_0)} \right]$$

Deoarece, pentru a aproxima cu precizia ϵ este necesară găsirea lui n minim astfel încât inegalitatea trecută să fie transformată în egalitate, putem exprima pe n necesar pentru a atinge precizia ϵ astfel:

$$n \sim \frac{1}{\log(p)} \log\left[\frac{\frac{\log(C_p)}{p-1} + \log(\epsilon)}{\frac{\log(C_p)}{p-1} + \log(e_0)} \right].$$

Dacă vom face N_p operații per iteratie, atunci totalul numărului de operații T_p poate fi exprimat ca $T_n = n \cdot N_p$ (numărul de iterații înmulțit cu numărul de operații făcute per pas), deci, n poate fi exprimat ca $\frac{T_n}{N_p}$.

Înlocuind în pasul anterior, numărul total de operații necesare pentru a obține precizia ϵ este:

$$T_p \sim \frac{N_p}{\log(p)} \log\left[\frac{\frac{\log(C_p)}{p-1} + \log(\epsilon)}{\frac{\log(C_p)}{p-1} + \log(e_0)} \right]$$

Subpunctul (b)

Ordinul de convergență al metodei secantei este dat de $p = \frac{1+\sqrt{5}}{2} \sim 1.61803\dots$ (sursa: curs)

$$p = (1 + \sqrt{5}) / 2$$

$$p = 1.61803398874989$$

Dacă aplicăm doi pași ai metodei secantei în același timp asta ar însemna că vom face 2 pași deodată:

$$e_{n+2} \sim C_p \cdot e_{n+1}^p \sim C_p (C_p \cdot e_n^p)^p = C_p^2 \cdot e_n^{p^2}$$

Deci un algoritm in care metoda secantei este aplicata de 2 ori la fiecare pas duce la cresterea patratica a eroarei asimptotice ($C_p \rightarrow C_p^2$), dar si la cresterea patratica a ordinului de convergenta ($p \rightarrow p^2$). Deci noul ordin de convergenta al algoritmului in care metoda secantei este aplicata de 2 ori per pas va fi:

$$p' = p^2 = \left(\frac{1 + \sqrt{5}}{2}\right)^2 = 2.61803\dots$$

```
pp=((1+sqrt(5))/2)^2
```

```
pp =  
2.61803398874989
```

Subpunctul (c)

Metoda lui Newton are ordinul de convergenta $p = 2$ si eroarea asimptotica C_2 . Fie un algoritm cu ordinul de convergenta $p = 3$ si eroarea asimptotica C_3 .

Numarul total de operatii facute de algoritmul lui Newton este:

$$T_2 = \frac{N_2}{\log(2)} \log \left[\frac{\frac{\log(C_2)}{2-1} + \log(\epsilon)}{\frac{\log(C_2)}{2-1} + \log(e_0)} \right] = \frac{N_2}{\log 2} \log \left[\frac{\log(C_2) + \log(\epsilon)}{\log(C_2) + \log(e_0)} \right]$$

Numarul total de operatii facute de algoritmul cu ordinul $p = 3$ este:

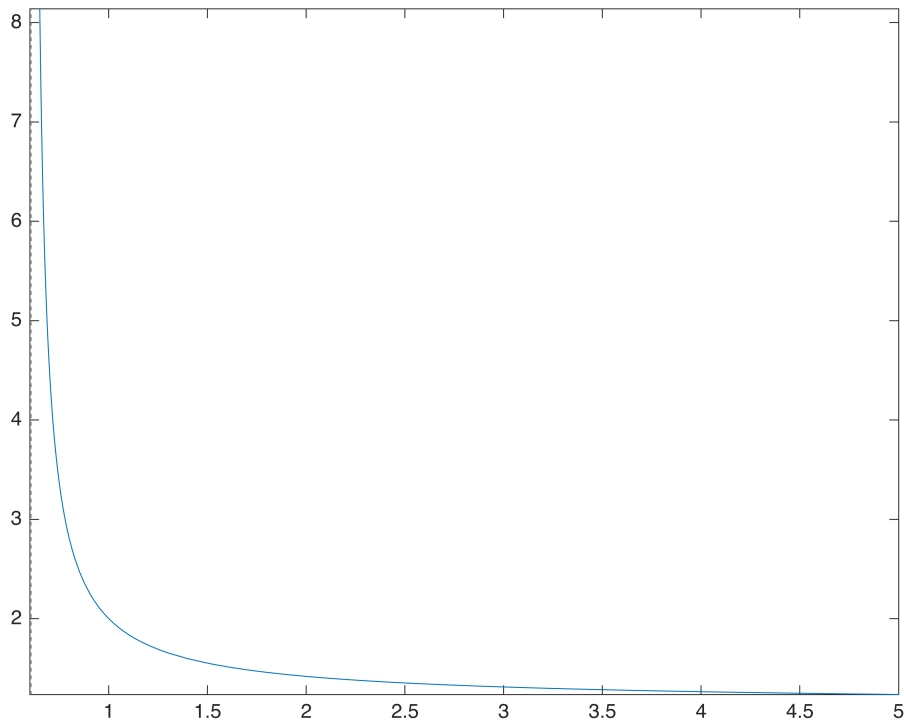
$$T_3 = \frac{N_3}{\log(3)} \log \left[\frac{\frac{\log(C_3)}{3-1} + \log(\epsilon)}{\frac{\log(C_3)}{3-1} + \log(e_0)} \right] = \frac{N_3}{\log(3)} \log \left[\frac{\log(C_3) + 2 \cdot \log(\epsilon)}{\log(C_3) + 2 \cdot \log(e_0)} \right]$$

Pentru ca metoda lui Newton sa fie mai eficienta decat metoda a doua, este necesar ca:

$$T_2 < T_3 \Leftrightarrow \frac{N_2}{\log 2} \log \left[\frac{\log(C_2) + \log(\epsilon)}{\log(C_2) + \log(e_0)} \right] < \frac{N_3}{\log(3)} \log \left[\frac{\log(C_3) + 2 \cdot \log(\epsilon)}{\log(C_3) + 2 \cdot \log(e_0)} \right].$$

Deoarece ϵ si e_0 sunt constante, ele nu vor influenta inegalitatea. Este clar ca $\epsilon < e_0$.

```
fplot(@(x) (log(x)+1)./(log(x)+0.5), [0.6 5])
```



Se observa ca o functie de tipul $\frac{\log(x) + a}{\log(x) + b}$, $a > b$ este descrescatoare, deci un $C_2 > C_3$ este mai favorabil. De asemenea, $N_2 < N_3$ ar ajuta algoritmul lui Newton sa fie mai rapid decat cel cu ordinul de convergenta $p = 3$. Deci, metoda lui Newton va fi mai eficienta decat o metoda cu ordinul de convergenta de complexitate $p = 3$ cand numarul de operatii per pas este mai mic ($N_2 \ll N_3$) si cand eroarea asimptotica devine mai mare ($C_2 \gg C_3$).

Problema 6. (a) Determinați spline-ul cubic deBoor pentru funcția $f(x) = \cos \pi x$ și diviziunea $\Delta : x_1 = -1 < x_2 = 0 < x_3 = 1$ (1p)

(b) De ce nu este un spline cubic propriu-zis? (1p)

Solutie

```
nodes=[-1 0 1]
```

```
nodes = 1x3
      -1    0    1
```

```
f=@(x) cos(pi*x);
values=f(nodes)
```

```
values = 1x3
      -1    1   -1
```

Trebuie sa gasim un spline cubic deBoor intr-o diviziune formata de $n=3$ puncte. Asta inseamna ca va trebui sa gasim 2 polinoame cubice ($p_i(x) = c_{i,0} + c_{i,1}x + c_{i,2}x^2 + c_{i,3}x^3$).

Problema spline-urilor este insuficienta ecuatiilor. Ele incearca sa egaleze nodurile de fixate cu valorile corespondente ($p_i(x_i) = f_i$, $i = 0 \dots n-1$, rezultand in $n-1$ ecuatii), dar si pentru nodurile din capatul drept (deci $p_i(x_{i+1}) = f_i$, $i = 0 \dots n-1$, rezultand tot in $n-1$ ecuatii). Pe langa acestea, ele si sa egaleze derivatele de ordinul 1 si 2 in punctele de jonctiune ($p_i'(x_{i+1}) = p_{i+1}'(x_{i+1})$, $p_i''(x_{i+1}) = p_{i+1}''(x_{i+1})$, $i = 0 \dots n-2$, rezultand in $2n-4$ ecuatii)

In total am obtinut $2n-2 + 2n-4 = 4n-6$ ecuatii, insa avem $4n-4$ necunoscute (4 pentru fiecare $p_{i=0..n-1}$). De aceasta putem defini mai multe tipuri de spline sa completeze acele conditii.

Spline-ul cubic deBoor are proprietatea ca:

$$p_1'''(x_2) = p_2'''(x_2) \text{ si } p_{n-1}'''(x_{n-1}) = p_{n-2}'''(x_{n-1})$$

Deoarece $n = 3$, problema este ca spline-ul deBoor in acest context adauga o singura informatie deoarece $n-1 = 2$ si $n-2 = 1$, deci a adaugat aceeasi ecuatie de 2 ori, deci vom avea un sistem subdeterminat (cu $4n-5$ ecuatii si $4n-4$ necunoscute), deci toate valorile vor depinde de o variabila. Scriem ecuatiile reiesite din spline-ul cubic deBoor:

```
syms p0 [1 4];
syms p1 [1 4];

eq01=p0(1) + p0(2)*nodes(1) + p0(3)*nodes(1)^2 + p0(4)*nodes(1)^3 ==
values(1), ...
eq02=p1(1) + p1(2)*nodes(2) + p1(3)*nodes(2)^2 + p1(4)*nodes(2)^3 ==
values(2)
```

$$eq01 = p_{01} - p_{02} + p_{03} - p_{04} = -1$$

$$eq02 = p_{11} = 1$$

```
eq11=p0(1) + p0(2)*nodes(2) + p0(3)*nodes(2)^2 + p0(4)*nodes(2)^3 ==
values(2), ...
eq12=p1(1) + p1(2)*nodes(3) + p1(3)*nodes(3)^2 + p1(4)*nodes(3)^3 ==
values(3)
```

```
eq11 =  $p_{01} = 1$ 
eq12 =  $p_{11} + p_{12} + p_{13} + p_{14} = -1$ 
```

```
eq2=p0(2) + 2*p0(3)*nodes(2) + 3*p0(4)*nodes(2)^2 == p1(2) +
2*p1(3)*nodes(2) + 3*p1(4)*nodes(2)^2
```

```
eq2 =  $p_{02} = p_{12}$ 
```

```
eq3=2*p0(3) + 6*p0(4)*nodes(2) == 2*p1(3) + 6*p1(4)*nodes(2)
```

```
eq3 =  $2p_{03} = 2p_{13}$ 
```

```
eq4=6*p0(4)==6*p1(4) % ecuatia adaugata de spline-ul deBoor
```

```
eq4 =  $6p_{04} = 6p_{14}$ 
```

```
[p01,p02,p03,p04,p11,p12,p13]=solve([eq01 eq02 eq11 eq12 eq2 eq3 eq4],
[p0(1),p0(2),p0(3),p0(4),p1(1),p1(2),p1(3)])
```

```
p01 = 1
p02 =  $-p_{14}$ 
p03 = -2
p04 =  $p_{14}$ 
p11 = 1
p12 =  $-p_{14}$ 
p13 = -2
```

```
syms alfa;
c=[np01 subs(np02, p1(4), alfa) np03 subs(np04, p1(4), alfa); np11
subs(np12, p1(4), alfa) np13 alfa]
```

```
c =
 $\begin{pmatrix} 1 & -\text{alfa} & -2 & \text{alfa} \\ 1 & -\text{alfa} & -2 & \text{alfa} \end{pmatrix}$ 
```

Observam ca variabilele rezultat depind de p_{14} .

```
function S=evalsplinecubic(x,nodes,values,c)
S=zeros(1, length(x));
for i=1:length(x)
    if x(i)==nodes(end)
        S(i)=values(end);
    else
        poz=find(nodes>x(i),1)-1;
        X=x(i);
```

```

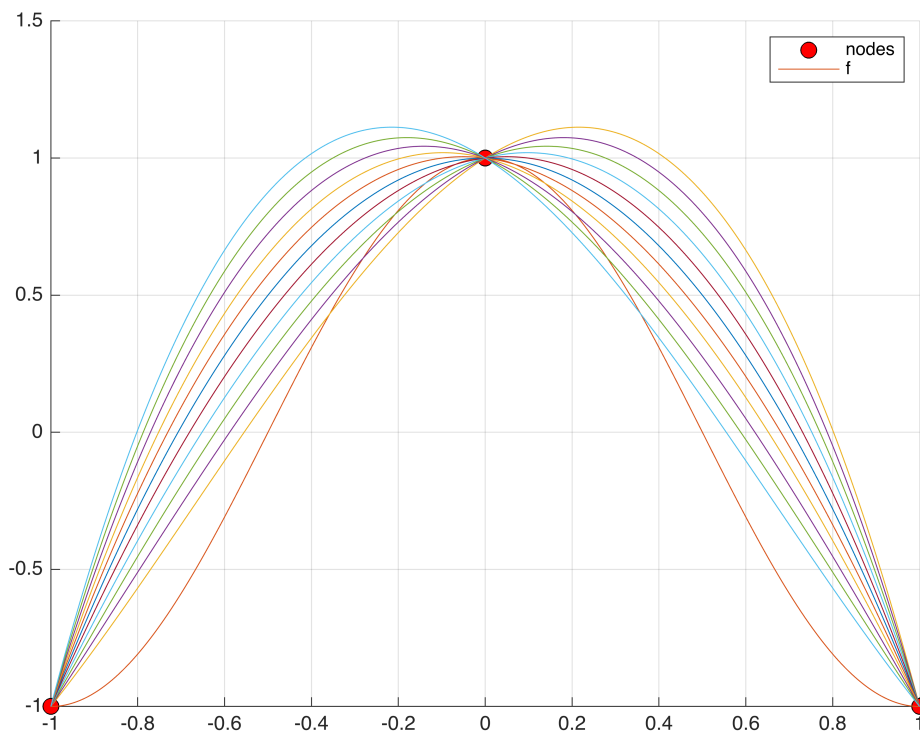
        S(i)=c(poz,1)+X*(c(poz,2)+X*(c(poz,3)+X*c(poz,4)));
    end
end
end

```

```

x=linspace(-1, 1, 1000);
clf; hold on; grid on;
plot(nodes, values, 'ok', 'markerfacecolor', 'r', 'markersize', 8)
plot(x, f(x))
for i=-1:0.2:1
    cn=double(subs(c,alfa,i));
    S=evalsplinecubic(x,nodes,values,cn);
    plot(x, S);
end
legend("nodes", "f")

```



Deci se observa ca nu obtinem doar o solutie, ci o infinitate, deci din acest motiv, nu este un spline cubic propriu zis.